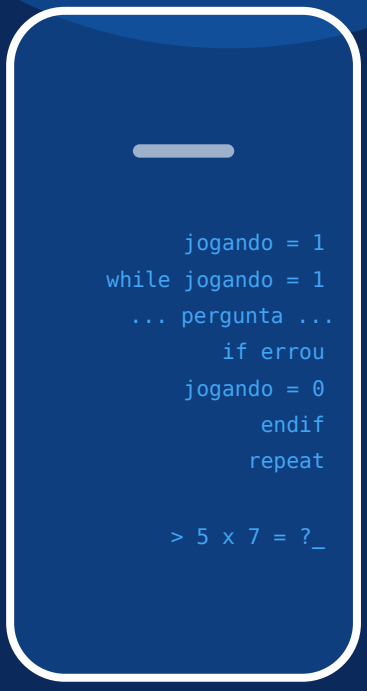
PROGRAMAÇÃO DE BOLSO RFO BASIC

AULA

04

Super Quiz



Um jogo que só termina quando você errar, e que fica mais difícil a cada acerto.

RECURSO DO CELULAR

Teclado e diálogos

laço while · condição de parada · variável sentinela · laços aninhados

Projeto de Ensino em Programação · Novembro de 2026

NESTA AULA VOCÊ VAI

- ✓ Repetir **enquanto** uma condição for verdadeira, com o laço `while`.
- ✓ Saber quando usar `for` e quando usar `while`.
- ✓ Controlar um laço com uma **variável sentinela**.
- ✓ Aumentar a dificuldade do jogo a cada acerto.
- ✓ Colocar um laço **dentro** de outro e criar um jogo com recorde.

Na Aula 3 o jogo tinha exatamente 10 perguntas, e o `for` era perfeito, porque sabíamos o número de voltas de antemão. Agora o desafio é outro: o jogo só acaba quando o jogador **errar**. Pode ser na primeira pergunta ou na quinquagésima. Ninguém sabe, nem o programa, e é por isso que precisamos de um laço novo.

1 O laço while

Listagem 1: Laço `while` com cinco iterações

`laco-while.bas` `basic`

```
1  n = 1
2
3  while n <= 5
4     print "n vale " + int$(n)
5     n = n + 1
6  repeat
7
8  print "Saí do laço com n = " + int$(n)
9  end
```

```
n vale 1
n vale 2
n vale 3
n vale 4
n vale 5
Saí do laço com n = 6
```

RODAR Execute. Por que o último valor impresso dentro do laço é 5, mas o `n` final é 6?

O `while` lê-se literalmente: "**enquanto** `n` for menor ou igual a 5, repita o que está até o `repeat`". A condição é testada **antes de cada volta**. Quando ela se torna falsa, o laço termina e o programa segue para a linha seguinte ao `repeat`.

CONCEITO · LAÇO WHILE

`while condição ... repeat` repete o corpo enquanto a condição for verdadeira. Como a condição é testada no **início**, se ela já começar falsa o corpo **nunca é executado**. E, para o laço terminar, alguma coisa **dentro dele** precisa mudar o resultado da condição.

Laço	Use quando…	Exemplo
`for`	você sabe quantas voltas serão	10 perguntas
`while`	a parada depende de **algo que acontece**	até o jogador errar

ATENÇÃO

Se nada dentro do laço altera a condição, ele **nunca termina**. Isso é um **laço infinito**. No exemplo acima, sem a linha `n = n + 1`, a condição `n <= 5` seria verdadeira para sempre. Para interromper o programa travado, use o botão voltar do aparelho.

2 Problemas que pedem while

Quantos dobros?

Quando usamos o `for` sabemos de antemão quantas voltas devem ocorrer no laço. Quando não temos essa informação, o `while` é a ferramenta certa. Por exemplo, quantas vezes é preciso dobrar o número 1 para ultrapassar 100?

Listagem 2: Laço `while` para calcular n tal que $2^n > 100$

`dobros.bas` `basic`

```
1  n = 1
2  vezes = 0
3
4  while n <= 100
5    n = n * 2
6    vezes = vezes + 1
7  repeat
8
9  print int$(vezes) + " dobros, chegando a " + int$(n)
```

O código entre `while` e `repeat` será executado se a condição `n <= 100` for verdadeira. Quando a condição for falsa, o laço acabará.

Volta	Condição	Valor de `n`	Valor de `vezes`
1	`1 <= 100?`: sim	1 x 2 = 2	0 + 1 = 1
2	`2 <= 100?`: sim	2 x 2 = 4	1 + 1 = 2

3	`4 <= 100?`: sim	4 x 2 = 8	2 + 1 = 3
4	`8 <= 100?`: sim	8 x 2 = 16	3 + 1 = 4
5	`16 <= 100?`: sim	16 x 2 = 32	4 + 1 = 5
6	`32 <= 100?`: sim	32 x 2 = 64	5 + 1 = 6
7	`64 <= 100?`: sim	64 x 2 = 128	6 + 1 = 7
8	`128 <= 100?`: **não**		

Quando chegar na sétima rodada, `n = 64` e `vezes = 6`. Como a condição é verdadeira, `n` vai dobrar e `vezes` recebe mais 1. Na próxima rodada a condição será falsa e o código do laço não será executado. Não dava para usar `for` aqui, pois não sabíamos o número de voltas.

RODAR Confira o resultado.

Note que a verificação da condição é no início do laço.

Soma dos dígitos

Como somar os dígitos de 4729 (4 + 7 + 2 + 9 = 22) **sem** tratar o número como texto? Com duas operações matemáticas:

- `mod(n, 10)` (resto da divisão de `n` por `10`) dá o **último dígito**;
- `floor(n / 10)` (seleciona a parte inteira de `n`) **remove** o último dígito.

Listagem 3: Laço `while` para calcular a soma de digitos

`digitos.bas` `basic`

```
1  n = 4729
2  soma = 0
3
4  while n > 0
5    soma = soma + mod(n, 10)
6    n = floor(n / 10)
7  repeat
8
9  print "Soma dos dígitos: " + int$(soma)
```

Volta	`n` no início	`mod(n, 10)`	`soma` no fim
1	4729	9	0 + 9 = 9
2	472	2	9 + 2 = 11
3	47	7	11+ 7 = 18
4	4	4	18 + 4 = 22

Depois da volta 4, `n` vira `floor(4 / 10) = 0`, a condição `n > 0` falha e o laço termina.

RODAR Troque o 4729 por outro número e antecipe a resposta com um teste de mesa antes de executar.

3 Super Quiz

Planejando como na aula passada:

Parte	Decisão
Repetição	`while jogando = 1`: continua enquanto o jogador não errar.
Estado	`acertos` (sequência atual) e `jogando` (1 = continua, 0 = acabou).
Se acertar	Soma 1 em `acertos` e continua.
Se errar	Muda `jogando` para 0, e o laço termina na próxima checagem.

CONCEITO · VARIÁVEL SENTINELA

A variável `jogando` **guarda a decisão** de continuar ou parar. O laço olha para ela a cada volta. Dentro dele, qualquer parte do programa pode "levantar a bandeira" (`jogando = 0`) para encerrar tudo. É uma técnica muito usada, porque deixa o código legível: "enquanto estiver jogando".

Listagem 4: Super Quiz

`super-quiz.bas` `basic`

```
1   acertos = 0
2   jogando = 1
3
4   while jogando = 1
5     a = floor(rnd() * 9) + 1
6     b = floor(rnd() * 9) + 1
7     input int$(a) + " x " + int$(b) + " = ", resp
8
9     if resp = a * b
10      acertos = acertos + 1
11      print "Acertou!"
12    else
13      print "Errou! Era " + int$(a * b)
14      jogando = 0
15    endif
16  repeat
17
18  print "Fim de jogo! Sua sequência: " + int$(acertos)
19  end
```

RODAR Jogue até errar. Quantas você acertou seguidas?

Dificuldade crescente

Com 9 como limite, o jogo cansa depressa. Podemos fazer o limite **crescer** junto com a sequência. Crie uma variável `limite`, comece com 9 e use-a nos sorteios:

Antes do `while`:

```
limite = 9
```

Dentro do `while`:

```
a = floor(rnd() * limite) + 1
b = floor(rnd() * limite) + 1
```

Dentro do `if`, depois de somar o acerto:

```
limite = 9 + 2 * acertos
```

Assim, depois de 5 acertos os números chegam até 19, depois de 10 até 29, e assim por diante.

4 Laço dentro de laço

Observe o código abaixo:

Listagem 5: Super Quiz com laço aninhado

super-quiz-rec.bas basic

```
recorde = 0
continuar = 1

while continuar = 1
  acertos = 0
  limite = 9
  jogando = 1

  while jogando = 1
    a = floor(rnd() * limite) + 1
    b = floor(rnd() * limite) + 1
    input int$(a) + " x " + int$(b) + " = ", resp

    if resp = a * b
      acertos = acertos + 1
      limite = 9 + 2 * acertos
    else
      jogando = 0
    endif
  repeat

  if acertos > recorde
    recorde = acertos
    msg$ = "Novo recorde: " + int$(recorde) + "!"
  else
    msg$ = "Sequência: " + int$(acertos) + "."
  endif

  print msg$
  dialog.message "Fim de jogo", "Jogar de novo?", escolha, "Sim", "Não"
  if escolha <> 1 then continuar = 0
repeat
end
```

Ele adiciona um recorde e a opção de jogar de novo. Isso exige **dois** laços: o de **dentro** controla uma partida, e o de **fora** controla se haverá outra. O laço de fora precisa **zerar** as variáveis da partida a cada recomeço.

```
Fim de jogo
Novo recorde: 7! Jogar de novo?

[ Sim ] [ Não ]
```

Veja como as responsabilidades se dividem:

Laço	Responsabilidade
`while continuar = 1`	Uma volta equivale a **uma partida inteira**.
`while jogando = 1`	Uma volta equivale a **uma pergunta**.
`recorde`	Criada **fora** de tudo, para sobreviver entre as partidas.
`acertos`, `limite`, `jogando`	Zeradas no início de cada partida.

ATENÇÃO

Lembra do experimento da Aula 3, em que colocar `acertos = 0` dentro do `for` zerava o placar a cada volta e estragava tudo? Aqui fazemos de propósito: cada partida **deve** recomeçar do zero. A diferença está no **laço** em que a linha mora.

5 Quebre o código!

QUEBRAR Apague a linha `jogando = 0` e execute. O que acontece? (Use o botão voltar para sair.)

QUEBRAR Coloque `jogando = 1` **dentro** do laço interno, logo depois do `while`. O jogo ainda termina quando você erra? Por quê?

QUEBRAR Troque `while jogando = 1` por `while jogando = 0`. O que o jogo faz? Por que o corpo nunca roda?

QUEBRAR Na versão final, apague `acertos = 0` do início da partida. O que acontece na segunda partida?

QUEBRAR Mova `recorde = 0` para dentro do laço externo. Ainda funciona o recorde? Por quê?

DICA

Quase todo erro de `while` cai em uma de duas famílias: a condição **nunca muda** (laço infinito) ou **já começa falsa** (laço que nunca roda). Treine o olho para as duas.

DESAFIO

Corrida contra o relógio

Em vez de parar no primeiro erro, o jogo dura **30 segundos**. Vale quantas contas o jogador acertar nesse tempo.

1. guarde o instante de início com `inicio = clock()`;
2. o laço continua enquanto `clock() - inicio < 30000`;
3. errar não encerra o jogo, mas **tira 1 ponto**;
4. no final, o celular fala o placar.

6 Resumo

Ideia ou comando	Para que serve
`while condição ... repeat`	Repete o corpo enquanto a condição for verdadeira.
Condição testada no início	Se começar falsa, o corpo nunca roda.
Laço infinito	Condição que nunca muda; botão voltar encerra.
Variável sentinela	Guarda a decisão de continuar (`jogando = 1`) ou parar (`0`).
`mod(n, 10)`, `floor(n / 10)`	Último dígito e remoção do último dígito.
Laços aninhados	O laço de dentro roda inteiro a cada volta do de fora.
`dialog.message`	Mostra uma caixa com botões e devolve o botão escolhido.
`clock()`	Relógio do aparelho, em milissegundos.

NA PRÓXIMA AULA

Aula 5: Sensor de nível. A grande estreia dos sensores. O celular vai descobrir se está de pé, deitado ou sendo chacoalhado. Usaremos o laço `do ... until` para **esperar** um evento, como “repita até chacoalhar”, e juntaremos tudo o que aprendemos até aqui.